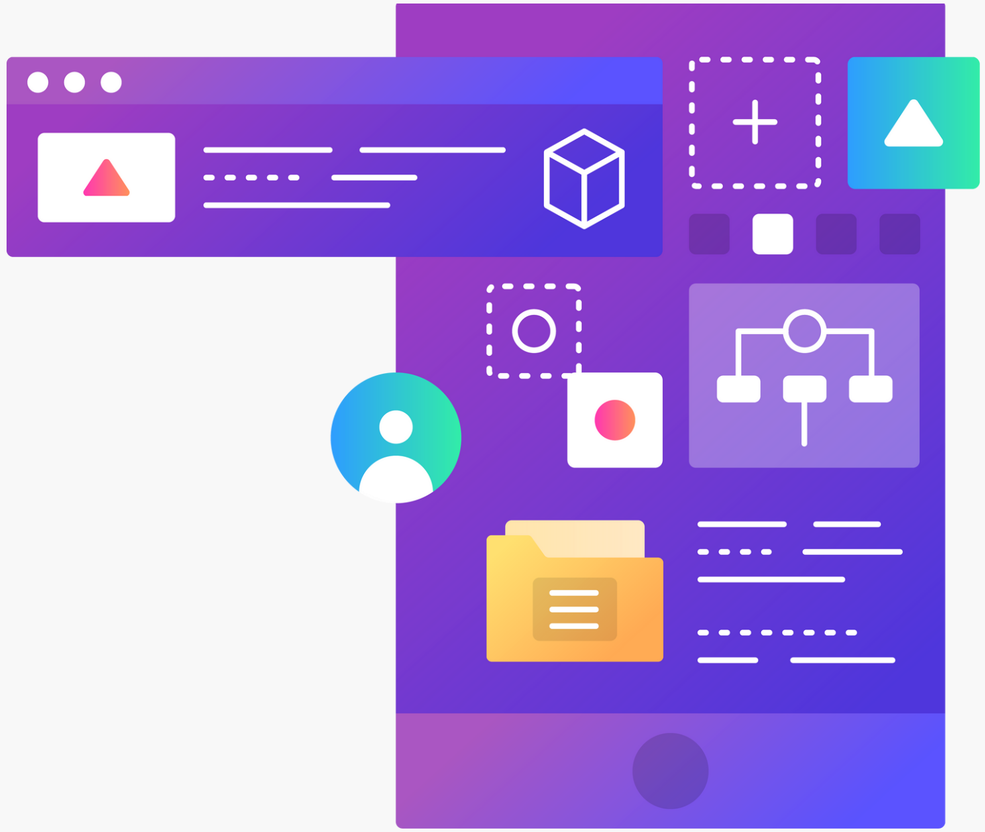
Chapter 4

Mobile Application Architecture

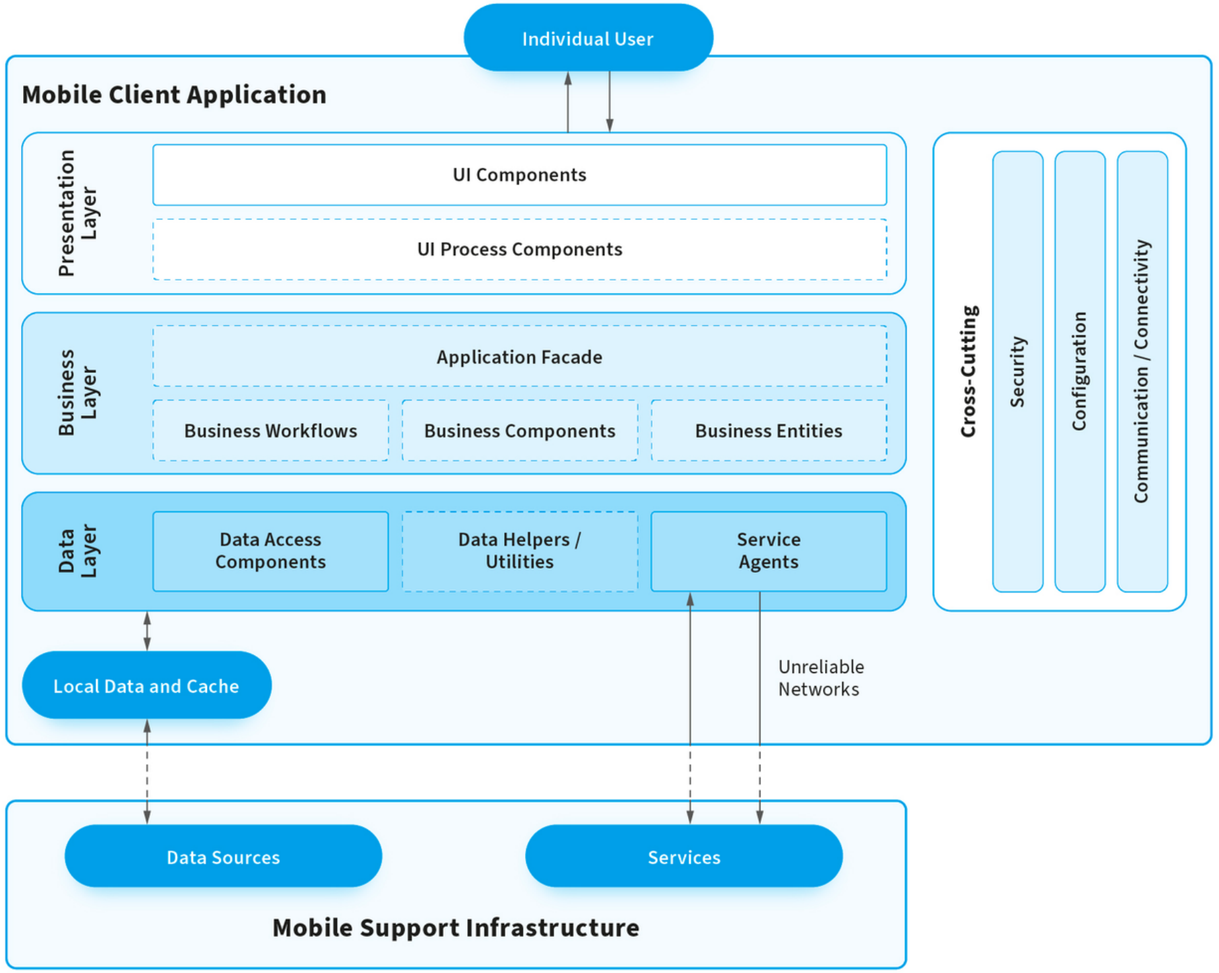
Hoang Le Hai Thanh
High Performance Computing Laboratory, HCMUT, VNU-HCM

Content

1. Mobile App Architecture Review
2. Separation of Concerns
3. MVP, MVC, MVVM
4. VIPER on iOS
5. Android App Architecture
6. MVI and React Redux
7. Mobile App Architecture - The Big Picture



Mobile App Architecture Review



The Presentation Layer

The presentation layer contains components for the user interface (UI), along with the components necessary for UI processing.

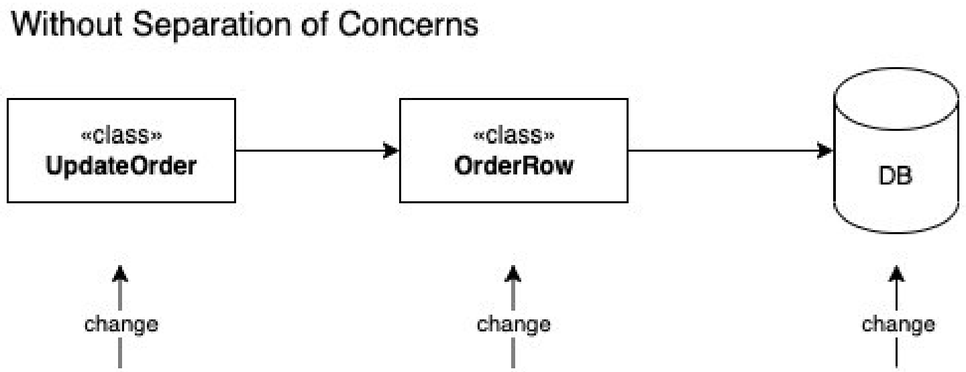
The Business Layer

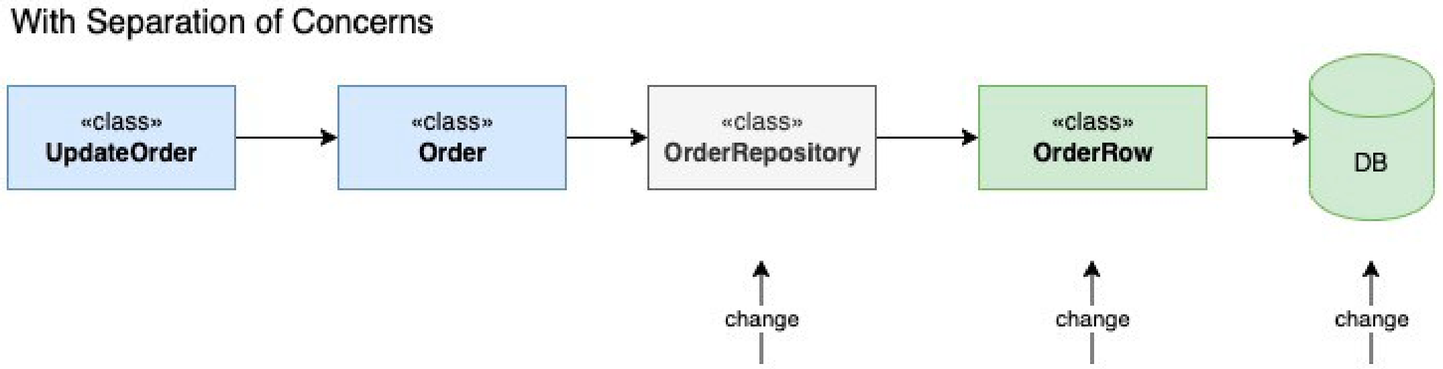
The business layer is concerned with the logic and rules responsible for data exchange, operations, and workflow regulation.

The Data Layer

The data layer includes all the data utilities, service agents, and data access components to support data transactions.

SEPARATION OF CONCERNS





Separation of Concerns (SoC) is a design principle for separating a mobile app into distinct sections such that each section addresses a separate concern:

- These layers should be loosely coupled
- Changes in one layer having limited or no impact on any other layers.

Single Responsibility vs Separation of Concerns

The Relationship between SRP and SoC

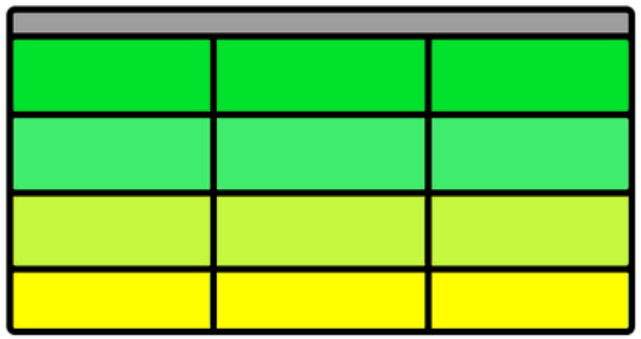
by Khalil Stemmler | @stemmlerjs

Single Responsibility Principle

A component should have one and one reason only to change. Based on Conway's Law, a component will only need to change if the role that utilizes the component determines change is necessary. Therefore, we should divide responsibility by role (vertically).

Single Responsibility (Role)

Admin Member Guest



Separation of Concerns

← API

← Data Access

← Application

← Domain

Separation of Concerns

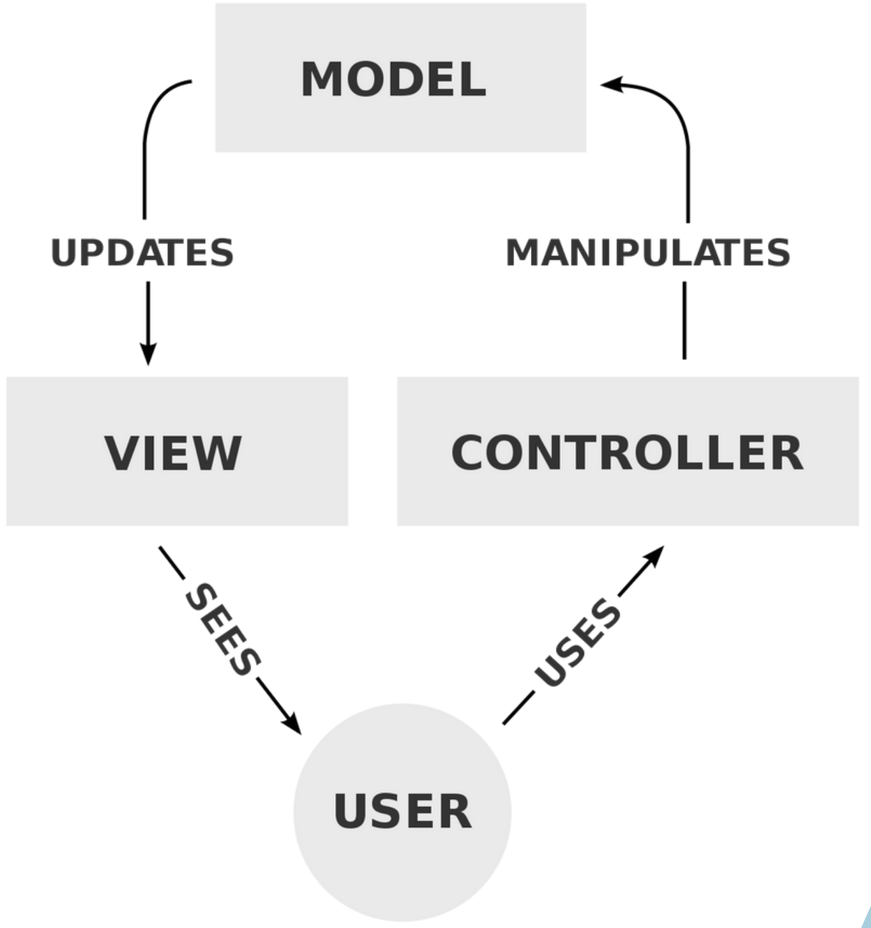
To realize a feature, we need to rely on a mix of infrastructure, application, and domain layer logic. The SoC principle urges us to divide the "technical" implementation details of a feature. To do this, we can implement a layered architecture (horizontal).

Feature = createPost

Model
View
Controller

Model - View - Controller pattern
(MVC) design pattern was introduced
by Trygve Reenskaug in the 1970s

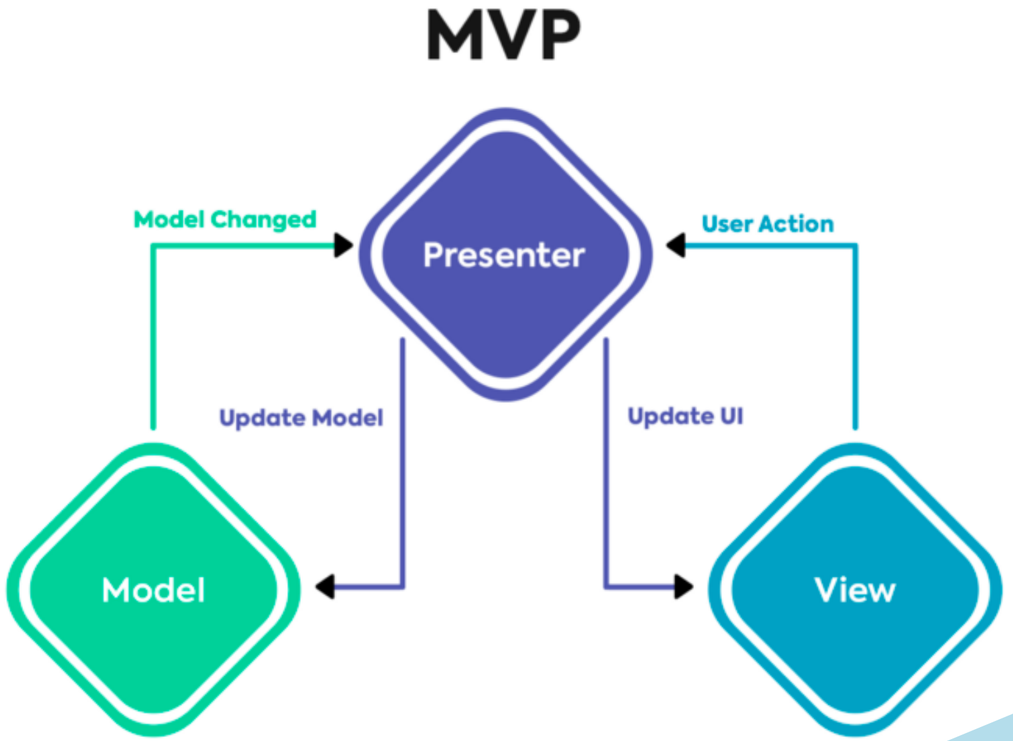
- The **model** is responsible for managing the data of the application. It receives user input from the controller.
- The **view** renders a presentation of the model in a particular format.
- The **controller** responds to the user input and performs interactions on the data model objects. The controller receives the input, optionally validates it, and then passes the input to the model.



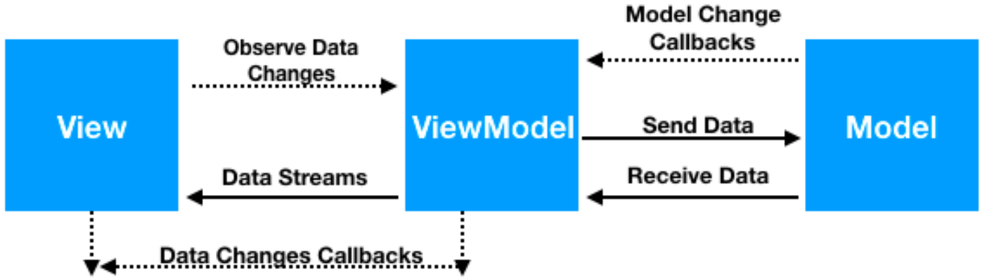
Model
View
Presenter

In MVP, the role of the controller is replaced with a Presenter.

- The **Model** represents a set of classes that describes the business logic and data. It also defines business rules for data means how the data can be changed and manipulated.
- The **View** represents the UI components. It is only responsible for displaying the data that is received from the presenter as the result. This also transforms the model(s) into UI.
- The **Presenter** is responsible for handling all UI events on behalf of the view. This receives input from users via the View, then process the user's data with the help of the Model and passes the results back to the View.



Model
View
ViewModel



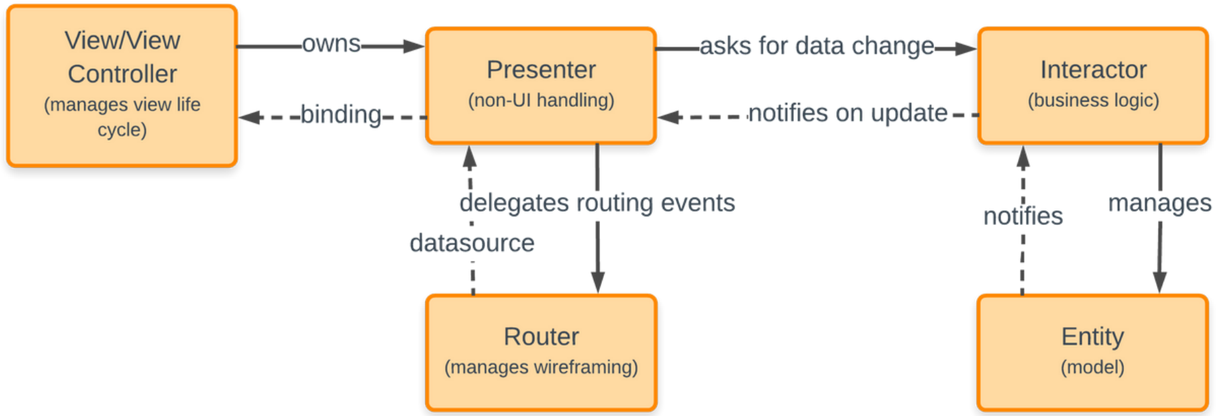
MVVM stands for Model-View-View Model. This pattern supports two-way data binding between view and View model.

- **Model:** This holds the data of the application. It cannot directly talk to the View. Generally, it's recommended to expose the data to the ViewModel through Observables.
- **View**: It represents the UI of the application devoid of any Application Logic. It observes the ViewModel.
- **ViewModel**: It acts as a link between the Model and the View. It's responsible for transforming the data from the Model. It provides data streams to the View. It also uses hooks or callbacks to update the View. It'll ask for the data from the Model.

VIPER (iOS)

VIPER is a backronym for View, Interactor, Presenter, Entity, and Router.

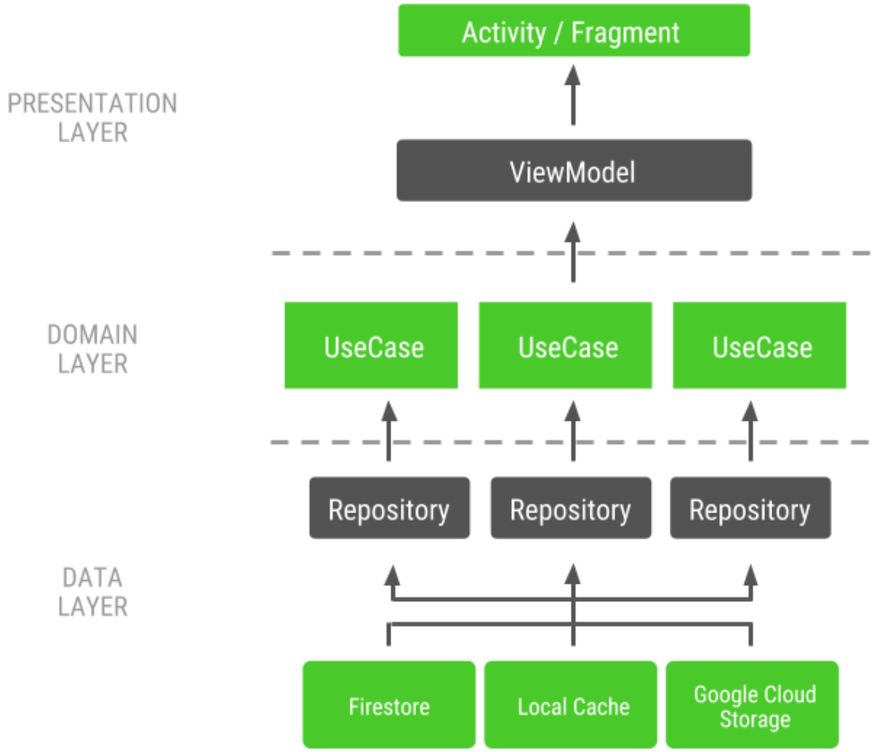
- **View(Controller):** this presents UI to the user and forwards the actions on UI to the Presenter to handle.
- **Presenter:** handles the UI actions sent by View. This handling is totally independent of the UI controls. In addition to this, Presenter takes the help of the Interactor to process the model(Entity) if need be.
- **Interactor**: mainly contains the business logic and integrates it with the other parts of the application. It also owns and manages the Entity.
- **Entity**: is just a dumb data structure to store data. As said, it is owned by the Interactor and concealed from the other modules.
- **Router**: is a special component in the VIPER architecture, which explicitly deals with the navigation actions coming from the Presenter. Navigation (Wireframing) is the only responsibility that the Router takes care of.



Android App Architecture

Based on Clean Architecture
The app is divided into a three-layer structure:

- **Presentation layer**: The role of the UI layer (or presentation layer) is to display the application data on the screen. Whenever the data changes, the UI should update to reflect the changes
- **Domain layer**: The domain layer is responsible for encapsulating complex business logic, or simple business logic that is reused by multiple ViewModels.
- **Data layer**: The data layer of an app contains the business logic. The business logic is what gives value to your app—it's made of rules that determine how your app creates, stores, and changes data.

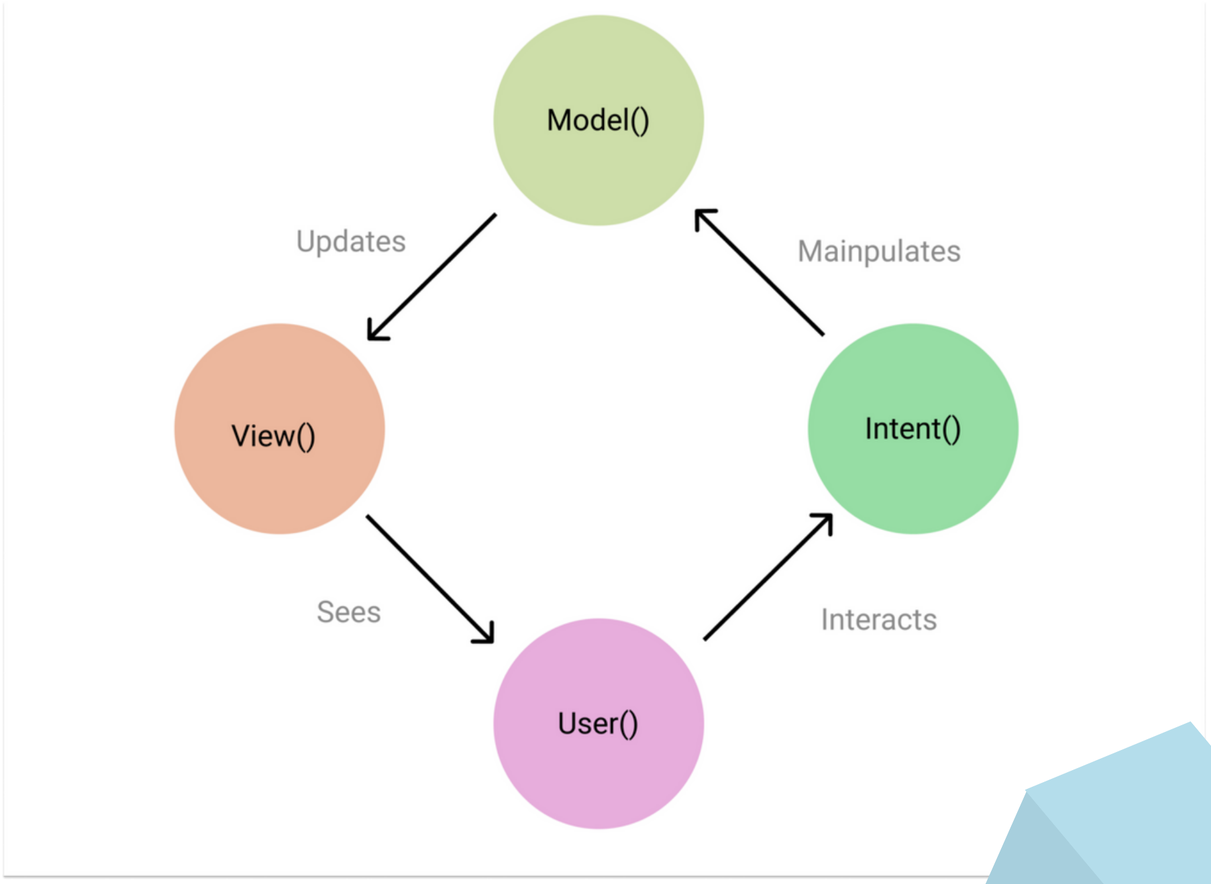


https://medium.com/androiddevelopers/google-i-o-2018-app-architecture-and-testing-f546e37fc7eb

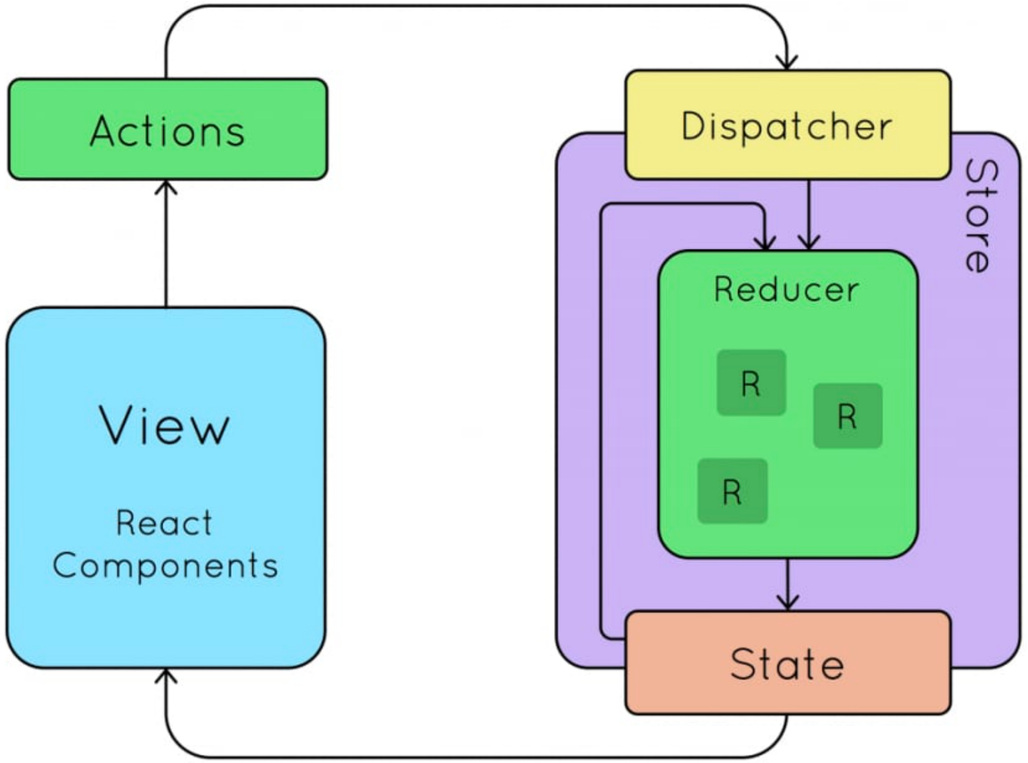
Model
View
Intent

MVI stands for Model-View-Intent. MVI imagines the user as a function, and models a unidirectional data flow based on that.

- **Model**- Other than representing the data and the business logic of the application model maintains a state which can be changed through the actions of the user.
- **View**- Represents the UI layer of the application which consists of Activities and Fragments. It accepts different model states and displays them as a UI.
- **Intent**- do not confuse it with the Android Intent, it's the intention to perform an action either by the user or the app itself.



React Redux



Store

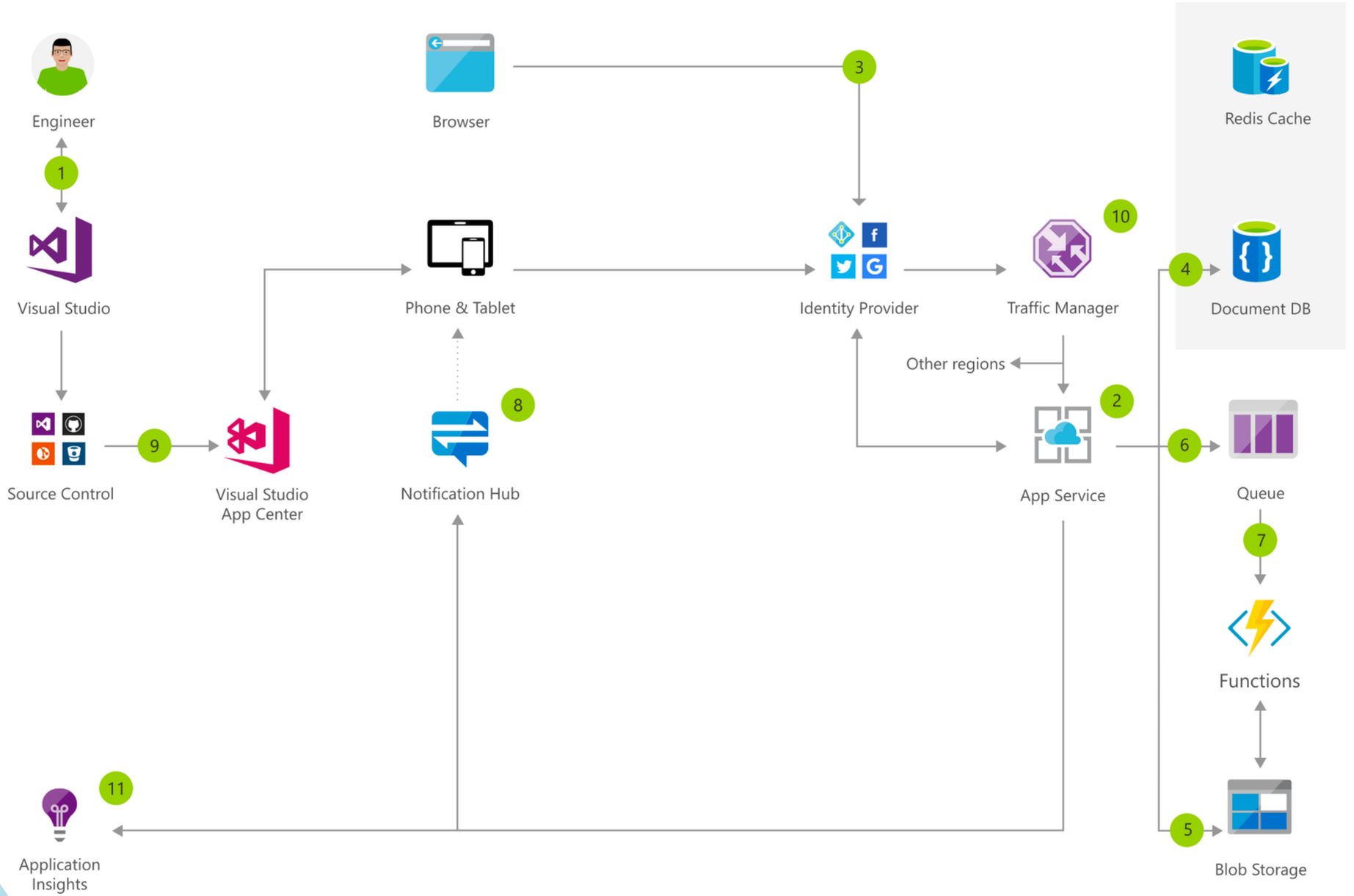
This is simply a state container that holds on to your app state. It holds an immutable reference representing the entire state of your application and can only be updated by dispatching an Action to it.

Actions

Actions contain information to be sent to the store. They represent how we want our state to be changed.

Reducers

Since neither Actions nor the Store update state themselves, we need a component dedicated to doing this. That's where Reducers come in. They simply take an Action and State and emit a new State.



THE BIG PICTURE

THANK YOU

Hoang Le Hai Thanh
High Performance Computing Laboratory
thanhhoang@hcmut.edu.vn